

ECHOES IN THE DARK

A First-Person Procedural Horror Experience in WebGL

Technical Report & User Manual

Interactive Graphics — Final Project
Sapienza University of Rome — Prof. Marco Schaerf

Team: Trio Monnezza	Matricola
Davide Timperi	1950722
Alexandru Vivian Pita	1948533
Federico Mendiola	1986026

[Live demo \(GitHub Pages\)](#)

1. Project Overview

Echoes in the Dark is a fully 3D, first-person survival-horror game built entirely with WebGL and Three.js, without relying on a pre-built game engine such as Unity or Unreal. It was developed as the final project for the Interactive Graphics course at Sapienza University of Rome.

The player wakes up trapped inside a dark, procedurally furnished building patrolled by a hostile, fully hand-animated monster. To escape, the player must explore the rooms, find a hidden golden key, and reach the exit door, all while managing a flashlight with a limited battery and avoiding — or fleeing from — the monster if it notices them. Three difficulty levels (Easy, Medium, Hard) offer progressively larger and more complex layouts with a proportionally larger number of rechargeable batteries scattered around the level.

The project intentionally leans into classic survival-horror tension: a single, precious light source that degrades over time, a monster that reacts believably to the player's presence, doors that can be used to break line of sight, and short cinematic sequences on death and on escape.

2. Environment and Tools

2.1 Technology Stack

The game runs entirely in the browser and is written in vanilla JavaScript (ES modules), with no build step and no game engine. It targets any modern browser with WebGL2 support.

Component	Choice
Rendering	Three.js r160 (WebGL2)
Language	Vanilla JavaScript, ES Modules (native browser import maps, no bundler)
Animation library	tween.js 21.0 — used for every procedural animation and UI transition in the project
Camera / input	Three.js PointerLockControls, extended with custom WASD movement, sprint and collision
Model loading	Three.js GLTFLoader (interior architecture and props, .glb) and FBXLoader (flashlight prop)
Audio	Web Audio API via Three.js AudioListener / Audio / PositionalAudio
Hosting	GitHub Pages (static hosting), served over a local dev server during development

2.2 Third-Party Libraries and Assets (Not Developed by the Team)

In line with the course requirements, no part of the game logic, animation, or level structure was imported from a pre-made source. The only external dependencies are:

- **Three.js (r160):** the WebGL rendering engine itself (scene graph, cameras, lights, materials, loaders). All meshes built from Three.js primitives, all animation, all game logic, and all collision handling are written by the team.

- **tween.js (21.0):** a generic interpolation/easing library. The team uses it as the animation engine for hand-written keyframe values — the specific animation curves, timings, and hierarchical rig logic are original work.
- **PointerLockControls, GLTFLoader, FBXLoader:** official Three.js example modules used only for their stated purpose (mouse-look input capture and mesh/scene import). FBXLoader pulls in fflate internally to decompress compressed FBX data; the team does not call fflate directly.
- **3D models and textures:** environment tiles, furniture props and the flashlight prop are .glb/.fbx assets sourced externally and assembled procedurally by the team's own placement code (see Section 4.10); none of them carry embedded animations, in compliance with the "no imported animations" rule — every motion in the game is computed at runtime by the team's own code.
- **Audio clips:** sound effects and ambient tracks are external .wav/.mp3 assets, played and mixed through a custom Web-Audio wrapper written by the team (Section 4.8).

3. Gameplay Mechanics

3.1 Player Movement and Stamina

Movement uses standard WASD (or arrow keys) with mouse-look, implemented on top of Three.js's PointerLockControls. Holding Space triggers a sprint that consumes a stamina resource shown as an on-screen bar; once stamina is depleted the player is forced back to walking speed until it partially regenerates. Collision against walls, doors and furniture uses axis-aligned bounding boxes with per-axis sliding resolution, so the player glides along a wall instead of stopping dead when approaching it at an angle.

3.2 Dynamic Flashlight

The flashlight is the player's only light source and is always active. Its battery drains gradually over time (roughly 1% every few seconds); as the charge drops, three things happen simultaneously and are all driven by the same underlying battery percentage: the beam's intensity dims, its cone angle and range physically narrow, and — once below a threshold — it begins to flicker with increasing frequency and depth the closer the battery gets to empty. A percentage indicator in the HUD keeps the player informed. Batteries scattered around the level (their count scales with difficulty) restore the charge to 100% when collected, using the same smooth transition curve that governs the drain.

On top of the battery system, the torch also sways procedurally: a gentle sinusoidal drift while standing still, a pronounced rhythmic bob while walking, an even wider one while sprinting, and — independently of movement — an erratic, jittering tremor that grows the closer the monster gets, meant to convey the character's rising panic.

3.3 The Monster

An invincible, hierarchically modelled creature patrols the level and actively hunts the player once it comes within its awareness radius. It cannot be fought — the player's only options are to run, break line of sight through a closed door, or simply not be noticed. Encountering it for the first time triggers a distinct startle animation; getting caught triggers a multi-phase lunge attack, after which the run ends with a cinematic death sequence (Section 4.9).

3.4 Interactive Doors

Every door in the level can be opened and closed with E. Doors are physical obstacles: while closed they block both movement and the monster's line of sight, letting the player use them tactically to break a chase. Some doors are locked and require the golden key; the exit door specifically requires it to complete the level.

3.5 Objective and Progression

The player must locate a hidden golden key hidden inside one of the rooms and bring it to the marked exit door to win. Three difficulty presets (Easy, Medium, Hard) provide progressively larger floor plans, different architectural themes (wood/wallpaper, plaster, stone/brick respectively) and a growing number of battery pickups (one, two and three respectively) to match the increased travel distance and risk.

4. Technical Architecture

This section documents how the project is actually implemented, file by file, as verified directly against the source in the submitted repository.

4.1 Rendering, Scene Setup and Level Generation

main.js owns the Three.js Scene, PerspectiveCamera and WebGLRenderer, wires together the game's controller objects, and runs the single requestAnimationFrame render loop that every other system hooks into.

Each difficulty (src/world/maps/MapEasy.js, MapMedium.js, MapHard.js) extends a shared MapBase class that assembles the level from a grid of typed cells: wall segments, floor/ceiling tiles, doors and pillars are instantiated as .glb tile pieces positioned procedurally from the grid coordinates, rather than being hand-placed one by one. MapBase also owns the axis-aligned bounding boxes used for collision (built directly while placing each tile — see Section 4.6), the interactive golden key and battery pickups (built from primitive Three.js geometry, not external models, and independently rotating/bobbing), and the trigger-zone system used for jump-scare and goal-reached events.

Furniture and decorative props are placed by src/world/InteriorAssetManager.js, which preloads and caches the .glb tile and prop library, and src/world/PropScale.js, which assigns each prop a per-asset uniform scale factor so its real-world footprint (in metres, converted through the game's own unit scale) is consistent with the player's eye height, rather than applying one flat scale multiplier to every object regardless of its real size.

4.2 Hierarchical Model: The Monster

src/entities/Monster.js builds the monster as a hand-assembled hierarchical Three.js Object3D graph — no rigged or pre-animated mesh is imported. The rig follows a classic articulated structure:

- Root → torso (corpo) → head (testa)
- Root → torso → left/right upper arm → left/right forearm (shoulder + elbow joints, mirrored)
- Root → left/right hip → left/right knee joint → shin → foot (a genuine knee pivot sits between thigh and shin, mirroring the arm's shoulder/elbow chain, so the leg bends at a real joint rather than swinging as one rigid piece)

Every limb is built from primitive Three.js geometries (boxes, cylinders) rather than imported meshes, satisfying the "complex hierarchical model" requirement while keeping the project free of any external animation data.

4.3 Procedural Animation System

All motion is computed at runtime in `src/animations/MonsterAnimator.js`, driven by `tween.js`, and is organised as a small state machine (`walk / idle / notice / attack`) layered on top of the hierarchy described above:

- **Walk cycle:** opposite-phase sinusoidal rotations on the shoulders, hips and the new knee joints (the knees oscillate at twice the hip frequency and with a phase offset between legs, so they visibly bend mid-stride instead of remaining rigid), plus a vertical torso bob and a forward predatory lean while chasing.
- **Idle:** a slow breathing motion plus unpredictable random "spasms" — an arm or the head snaps briefly out of position at randomised intervals — to avoid a perfectly looping, mechanical idle pose.
- **Notice ("shock"):** a short, sharp startle animation (head snap, torso recoil, arms jerking up) fired the exact frame the monster's AI first detects the player, rather than a generic escalation.
- **Attack:** a three-phase lunge — anticipation (wind-up, crouch), strike (a diagonal claw swing with a forward lunge of the torso and head), and recoil (a slower return to neutral) — chained tween-to-tween so the whole motion is one continuous, uninterrupted curve.

A recurring engineering rule enforced throughout this animation code is that no system ever writes to a Three.js property owned by another system in the same frame: for example, the monster's forward lean is applied to the torso's own local rotation rather than to the root object, because the root's rotation is exclusively driven every frame by `Object3D.lookAt()` (used to keep the monster facing the player) — writing to both from two different systems on the same object produces visibly incorrect orientations, since a quaternion's Euler decomposition is not unique. The same discipline governs the flashlight sway, the screen-shake effects and the camera FOV kick described below: each animated property has exactly one owner.

4.4 Monster AI

`src/core/MonsterAI.js` implements the decision-making layer that drives the animation state machine above. Each frame it:

- computes the straight-line distance to the player and, the instant that distance first drops below the aggro radius, fires the one-shot "notice" event described above;
- while the player is out of range, periodically relocates the monster to a room or doorway near the player if too much time has passed without contact, so the threat never becomes purely avoidable by distance alone;
- while chasing, casts a small fan of rays in front of the monster to steer around obstacles, and treats a low-scoring, cornered situation (little real displacement over time) as "stuck", first attempting a lateral escape direction and, if that also fails, relocating the monster near the player as a last resort — this prevents the AI from getting permanently wedged in geometry;
- can open a nearby closed door itself when patrolling toward the player;
- only commits to the lunge/attack sequence — and only actually resolves it into a game-over — if a raycast line-of-sight check confirms no closed door or wall stands between the monster and the player, so the monster cannot "kill through" a door the player has just shut; the AI keeps chasing without attacking until it can path to a position with a clear line of sight;
- keeps the actual game-over event decoupled in time from the start of the attack animation, so the death is resolved at the animation's impact frame rather than the instant the lunge begins.

The monster's movement itself resolves collisions the same way the player's does (Section 4.6): a fan of forward probe rays picks the least-obstructed direction, and the final step is still validated per-axis against the level's bounding boxes before being applied, so the monster cannot clip through walls or doors even if the steering heuristic picks a bad direction.

4.5 Collision System

Collision uses axis-aligned bounding boxes (`THREE.Box3`), registered through `MapBase`'s `addInvisibleCollisionBox()` helper directly while each wall/door/prop tile is instantiated, and tested every frame against the player's and the monster's own bounding volumes. Movement resolution is per-axis (X and Z tested and applied independently), which produces the smooth "sliding along a wall" behaviour expected of a first-person controller rather than a hard stop on any oblique contact. Door collision boxes are dynamically emptied while a door is open — cleared only once it is visually ajar enough to plausibly walk through, not the instant the animation starts — and restored the moment it swings shut, so both the player and the monster are physically blocked by a closed door and can pass freely through an open one. A separate `src/world/CollisionBuilder.js` class documents an alternative box-registration API but is not wired into the running game; it can be considered dead code.

4.6 Lighting and Texturing

The scene combines a low-level ambient light and a faint directional light for base visibility with per-object dynamic lights: the player's flashlight is a `THREE.SpotLight` (plus a small supporting `THREE.PointLight` attached to the flashlight prop itself), and individual rooms place additional point lights, some of which flicker procedurally for atmosphere.

Wall, floor and ceiling materials (`src/world/TextureLoader.js`) use `THREE.MeshPhongMaterial` with a colour/diffuse map paired with a normal map for surface relief and a specular map for per-material shininess (e.g. a wet floor reflects highlights differently than a dry one); some surfaces intentionally omit the specular map where a flat, non-reflective look is wanted. Three distinct material sets (wood/wallpaper, plaster/tile, stone/brick/concrete) are used across the three difficulty levels to visually distinguish them. Procedural, non-textured objects built directly in `MapBase.js` — the pickup key, battery props, and a handful of fallback materials — instead use `THREE.MeshStandardMaterial`, the PBR (physically-based) workflow; the project therefore uses both lighting models side by side, each where it fits best, rather than one uniformly.

4.7 Audio System

`src/core/AudioSystem.js` wraps the Web Audio API (through `Three.js`'s `AudioListener/Audio/PositionalAudio`) into a small centralised service: a looping background track that smoothly cross-fades between an "ambience" and a more intense "pursuit" mix depending on whether the player is within the monster's aggro range, one-shot UI/interaction sounds, footsteps sounds tied to the player's movement state, and positional 3D sounds (e.g. the monster's breathing, door creaks and slams) that are actually emitted from their 3D world position, so the player can judge distance and direction by ear.

4.8 Death and Victory Sequences

Rather than cutting to a static end screen, both outcomes play a short scripted sequence built with `tween.js` and plain CSS transitions, orchestrated from `src/ui/GameUIController.js`:

- **Death:** on-screen blood splatter shapes are generated procedurally — organic blobs built from randomised, irregular border-radius values combined with a radial gradient for a wet highlight and CSS box-shadow layers for scattered droplets, so no two deaths render identical marks — the screen then fades to black before the game-over text fades in.
- **Victory:** the exit door animates open and shrinks away, a warm light overlay blooms across the screen together with softly rising, fading particle "sparks", and the screen fades to a warm white before the escape text fades in — a deliberate visual mirror of the death sequence.

4.9 AI-Assisted Room Decoration

To populate the already-built rooms with believable furniture layouts, the team used an LLM assistant (Claude) as a level-decoration tool rather than a code generator: the finished structural maps were provided as context, and the assistant was instructed to place only existing prop models from the team's own asset library into the existing rooms, respecting real-world proportions, leaving walking space clear, avoiding overlapping collisions, and keeping each room thematically coherent (e.g. grouping bathroom fixtures together, cluttering a storage room). The resulting placement data is consumed by `InteriorAssetManager.js` exactly like any other prop instance — the assistant did not write or modify any game logic.

5. User Interaction Summary

The project satisfies the course's user-interaction requirement through several independent, functionally meaningful forms of input, summarised here and detailed further in the User Manual (Section 8):

Interaction	Effect
WASD / Arrow keys	First-person movement, with sliding collision against the environment
Mouse	Look around (pointer-lock camera control)
Space (hold)	Sprint; drains a stamina resource with a visible cooldown
E	Context-sensitive interaction: open/close a door, pick up the key or a battery
Menu buttons	Choose difficulty level, restart, return to main menu, adjust volume
G (debug)	Toggles a developer "safe mode" in which the monster still detects, chases and "attacks" the player, playing every animation, but cannot end the run — used for testing and demonstration, off by default

6. Repository Structure

```
final-project-triomonnezza/
|-- assets/                3D models (.glb/.fbx), textures, audio
|-- lib/                   Vendored copies of Three.js, tween.js, etc.
|-- src/
|   |-- animations/       MonsterAnimator.js, TweenManager.js
|   |-- core/             PlayerController, MonsterAI,
|   |                     DoorController, FlashlightController,
|   |                     AudioSystem
|   |-- entities/         Monster.js (hierarchy construction)
|   |-- ui/               GameUIController.js, BloodSplatter.js
|   |-- world/            MapBase / MapEasy / MapMedium / MapHard,
|   |                     InteriorAssetManager.js, PropScale.js,
|   |                     TextureLoader.js
|-- index.html / index.css Entry point and UI styling
|-- main.js               Scene bootstrap and render loop
`-- download_libs.py      Fetches vendored library copies
```

7. Academic Requirements Compliance

Requirement	How it is met
WebGL / approved library	Three.js, used throughout for rendering, scene graph and loaders only
Hierarchical model	The monster: a multi-level Object3D parent/child rig (torso → head/arms/forearms, hips → knees → shins → feet), built entirely from code
Animations not imported	Every animation (monster locomotion/attack, doors, camera/flashlight effects, UI transitions) is computed at runtime with tween.js and hand-written easing/keyframe values; no .glb/.fbx asset carries embedded animation data
Lights and textures	Ambient, directional, spot and point lights; diffuse, normal and specular maps on environment materials
User interaction	Movement, sprint/stamina, flashlight-driven visibility, door interaction, difficulty selection, menus (Section 5)
Physics / collision (optional)	Custom AABB collision with per-axis sliding, used for both the player and the monster; no external physics engine

As required, the accompanying repository contains the full source code (no build artefacts are needed, since the project runs directly as ES modules) and this document, and the live build is published via GitHub Pages, linked from the repository's README.

8. User Manual

8.1 Playing Online

The simplest way to play is the published build:

sapienzainteractivegraphicscourse.github.io/final-project-triomonnezza. No installation is required — open the link in any modern desktop browser (Chrome, Firefox or Edge recommended) with WebGL2 enabled.

8.2 Running Locally

Because the game loads external assets (models, textures, audio) via `fetch()`, it must be served over HTTP — opening `index.html` directly from disk will fail due to browser CORS restrictions.

- Clone the repository: `git clone <repository-url> && cd final-project-triomonnezza`
- (Optional) fetch vendored library copies: `python download_libs.py`
- Start any static server from the project root, for example: `python -m http.server 8000`
- Open `http://localhost:8000` in the browser.

8.3 Controls

Input	Action
W / A / S / D or Arrow keys	Move forward / left / backward / right
Mouse	Look around
Space (hold)	Sprint (consumes stamina)
E	Interact — open/close the nearest door, pick up the key or a battery
G	Toggle developer safe-mode (off by default; see Section 5)

8.4 Objective

Explore the level, find the hidden golden key, and carry it to the marked exit door to escape. A HUD indicator confirms once the key has been collected. Avoid — or hide from — the monster: if it catches you, the run ends and you may retry.

8.5 Difficulty Levels

Level	Layout	Batteries
Easy	Smaller floor plan, wood/wallpaper interior	1
Medium	Mid-sized floor plan, plaster/tile interior	2
Hard	Largest, most complex floor plan, stone/brick/concrete interior	3

Difficulty is chosen from the main menu before starting a run and can be changed at any time via Restart from the pause/game-over screen.

8.6 Survival Tips

- Keep an eye on the flashlight battery percentage in the HUD — a narrowing, flickering beam means it is critically low; a battery pickup fully restores it.
- A closed door blocks the monster's line of sight as well as its path — use doorways to break a chase, but remember the monster can open unlocked doors itself if it reaches one.
- The torch's own jitter is a tell: the more erratically it shakes and the harder it flickers with the monster nearby, the closer it actually is, even before you see it.
- Sprinting is fast but not free — running out of stamina at the wrong moment can be fatal; pace yourself when being chased.